

Universidade Federal de Juiz de Fora
Departamento de Ciência da Computação
Gerência de Dados

Modelagem de Dados Híbrida para um Sistema de Gerenciamento de Pedidos Online em Larga Escala

Camila Corrêa Vieira

Professor: Victor Ströele

Trabalho final de Gerência de Dados, parte integrante da avaliação da disciplina.

Juiz de Fora

12 de Setembro de 2025

1 Introdução

O presente trabalho parte de um cenário hipotético de uma empresa de comércio eletrônico, denominada "CamiEcom", em fase de expansão acelerada. A arquitetura de dados legada, caracterizada por uma abordagem monolítica e dependente de um único sistema de gerenciamento de banco de dados, foi identificada como um gargalo que impõe limitações à escalabilidade e ao desempenho operacional. Diante disso, o objetivo central deste trabalho é projetar e avaliar uma nova arquitetura de dados, fundamentada no conceito de que seja capaz de suportar um volume massivo de transações e usuários, ao mesmo tempo que habilita capacidades de análise de dados em tempo real.

- Para delimitar o escopo da solução, foram definidos os seguintes requisitos funcionais essenciais que o sistema proposto deve atender:

Codigo	Identificacao	Classificacao	Ator	Objetivo
[RF001]	Manter Produto	Essencial	Administrador	Este caso de uso serve para que o administrador possa gerenciar (criar, ler, atualizar e apagar) os produtos da loja.
[RF002]	Manter Cliente	Essencial	Cliente	Este caso de uso serve para que o cliente possa se cadastrar e manter suas informações pessoais e de endereço.
[RF003]	Realizar Pedido	Essencial	Cliente	Este caso de uso serve para que o cliente possa selecionar produtos e finalizar uma compra.
[RF004]	Consultar Histórico de Pedidos	Importante	Cliente	Este caso de uso serve para que o cliente possa visualizar uma lista de todos os pedidos que já realizou.
[RF005]	Consultar Detalhes do Pedido	Importante	Cliente	Este caso de uso serve para que o cliente possa ver todas as informações de um pedido específico, como produtos, valores e status da entrega.

continua na próxima página

<i>continuação da página anterior</i>				
Codigo	Identificacao	Classificacao	Ator	Objetivo

Tabela 1: Tabela de Requisitos Funcionais

- Adicionalmente aos requisitos funcionais, foram estabelecidos os seguintes requisitos não-funcionais (RNFs) para garantir que a solução atenda aos critérios de qualidade, desempenho e escalabilidade esperados.

Codigo	Categoria	Requisito	Metrica / Criterio de Aceite
[RNF001]	Escalabilidade	O sistema deve ser capaz de escalar horizontalmente para suportar o crescimento do negócio.	Suportar um aumento de 100x no número de pedidos e clientes nos próximos 2 anos sem degradação do desempenho.
[RNF002]	Desempenho	O tempo de resposta para consultas críticas do cliente deve ser mínimo para garantir uma boa experiência de usuário.	A consulta de detalhes de um pedido e do histórico de um cliente deve ser realizada em menos de 200ms em 95% das requisições.
[RNF003]	Disponibilidade	O sistema de recebimento de novos pedidos deve estar operacional na maior parte do tempo para evitar perdas de vendas.	O endpoint de criação de pedidos deve ter uma disponibilidade de 99,95% (alta disponibilidade).
[RNF004]	Observabilidade/ Análise	A gerência de negócios precisa de visibilidade sobre as operações de venda para tomada de decisão ágil.	Um painel de monitoramento deve exibir o volume de vendas com uma latência máxima de 5 segundos em relação ao evento real.

Tabela 2: Tabela de Requisitos Não-Funcionais

2 Metodologia utilizada

2.1 Modelo Relacional (PostgreSQL)

Para a estruturação do banco de dados, foi adotado o modelo relacional com uma estratégia de normalização, implementado em PostgreSQL. Esta abordagem foi selecionada por sua robustez em garantir a consistência e a integridade dos dados, características essenciais para sistemas transacionais (OLTP). A modelagem segmenta os dados em entidades distintas para minimizar a redundância. As entidades centrais do sistema são:

- Clientes (id_cliente, nome, email, senha e data_cadastro)
- Produtos (id_produto, nome, preco, descrição e estoque.)
- Pedidos (id_pedido, id_cliente, data_pedido, status, endereco_entrega)
- Itens_Pedido tabela de junção para relacionamento N-para-N entre Pedidos
- Produtos, contendo id_pedido, id_produto, quantidade, preco_unitario).

Script DDL (SQL) para criação de todas as tabelas e seus relacionamentos:

```
— Tabela para armazenar os dados dos clientes
CREATE TABLE Clientes (
    id_cliente SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL,
    email VARCHAR(255) NOT NULL UNIQUE,
    senha VARCHAR(255) NOT NULL,
    data_cadastro TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP
);

— Tabela para armazenar os dados dos produtos
CREATE TABLE Produtos (
    id_produto SERIAL PRIMARY KEY,
    nome VARCHAR(255) NOT NULL,
    descricao TEXT,
    preco DECIMAL(10, 2) NOT NULL CHECK (preco > 0),
    estoque INTEGER NOT NULL DEFAULT 0 CHECK (estoque >= 0)
);

— Tabela para armazenar os pedidos dos clientes
CREATE TABLE Pedidos (
```

```

    id_pedido SERIAL PRIMARY KEY,
    id_cliente INTEGER NOT NULL,
    data_pedido TIMESTAMP WITH TIME ZONE DEFAULT CURRENT_TIMESTAMP,
    status VARCHAR(50) NOT NULL DEFAULT 'Pendente',
    endereco_entrega TEXT NOT NULL,
    FOREIGN KEY (id_cliente) REFERENCES Clientes(id_cliente)
    ON DELETE CASCADE
);

```

— *Tabela de juncao para o relacionamento*

—*N-para-N entre Pedidos e Produtos*

```

CREATE TABLE Itens_Pedido (
    id_pedido INTEGER NOT NULL,
    id_produto INTEGER NOT NULL,
    quantidade INTEGER NOT NULL CHECK (quantidade > 0),
    preco_unitario DECIMAL(10, 2) NOT NULL,
    PRIMARY KEY (id_pedido, id_produto),
    FOREIGN KEY (id_pedido) REFERENCES Pedidos(id_pedido)
    ON DELETE CASCADE,
    FOREIGN KEY (id_produto) REFERENCES Produtos(id_produto)
    ON DELETE RESTRICT
);

```

/*

Comentarios sobre o design:

1. SERIAL e usado para chaves primarias auto-incrementais no PostgreSQL.
2. UNIQUE no email do cliente previne cadastros duplicados.
3. CHECKs sao usados para garantir a validade dos dados numericos (precos e estoque positivos).
4. ON DELETE CASCADE em Pedidos(id_cliente) garante que, se um cliente for removido, todos os seus pedidos tambem serao.
5. ON DELETE RESTRICT em Itens_Pedido(id_produto) previne que um produto seja deletado se ele estiver associado a algum pedido.

*/

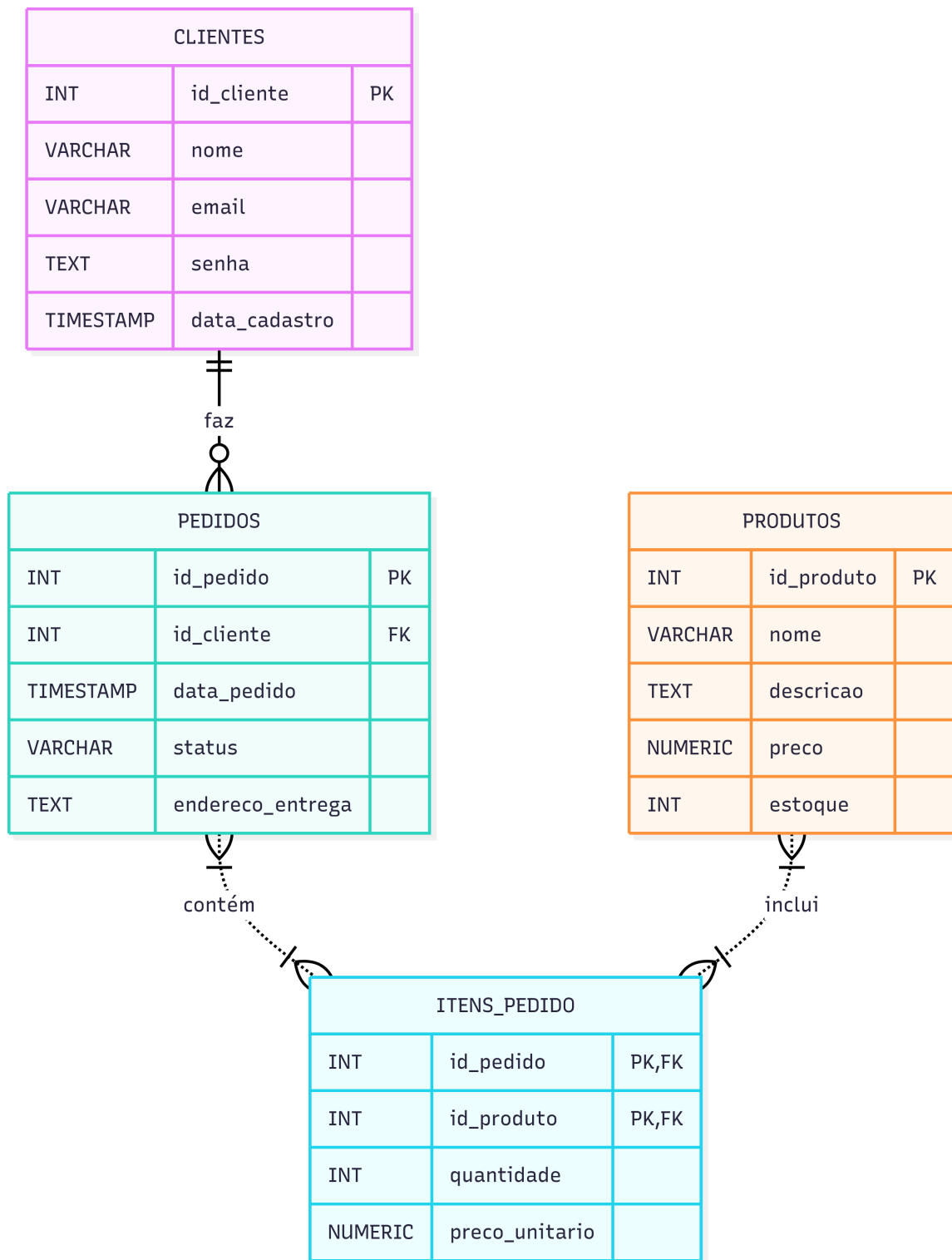


Figura 1: Diagrama Entidade-Relacionamento (DER).

Como apresentado na Figura 1, os dados foram segmentados em entidades distintas (clientes, produtos, pedidos), eliminando a redundância e garantindo a integridade referencial através de chaves estrangeiras. Por exemplo, o nome de um cliente é armazenado em um único local na tabela clientes. Se o nome precisar ser atualizado, a alteração é feita em um só lugar, garantindo consistência em todo o sistema. A relação muitos-para-muitos entre pedidos e produtos exigiu a criação de uma tabela de junção, itens_pedido, para manter a forma normal. Esta abordagem prioriza a consistência e a integridade dos dados (ACID).

2.2 Modelo NoSQL (MongoDB)

Para otimizar as operações de leitura, foi projetado um esquema desnormalizado, ideal para bancos de dados orientados a documentos como o MongoDB. A estratégia principal é incorporar dados relacionados dentro de um único documento, eliminando a necessidade de operações de JOIN para buscar as informações de um pedido completo.

A estrutura se baseia em uma coleção principal chamada pedidos. Cada documento nesta coleção representa um único pedido e agrega todas as informações relevantes, incluindo dados do cliente e os itens comprados. Essa modelagem acelera a recuperação de dados para visualização de pedidos, uma operação comum em sistemas de e-commerce.

Abaixo está um exemplo de como um documento de pedido seria estruturado na coleção pedidos.

```
{
  "_id": "631a0b78e4b0c8a2f3f4e5d6",
  "data_pedido": "2025-09-15T14:30:00Z",
  "status": "Processando",
  "cliente": {
    "id_cliente_ref": 101,
    "nome": "Ana_Souza",
    "email": "ana.souza@example.com"
  },
  "itens": [
    {
      "id_produto_ref": 789,
      "nome_produto": "Smartphone_XYZ",
      "quantidade": 1,
      "preco": 2500.00
    },
    {
```

```

      "id_produto_ref": 456,
      "nome_produto": "Capa_Protetora",
      "quantidade": 1,
      "preco": 89.90
    }
  ],
  "endereco_entrega": {
    "logradouro": "Avenida_Principal, 123",
    "cidade": "Rio_de_Janeiro",
    "estado": "RJ",
    "cep": "20000-000"
  },
  "valor_total": 2589.90
}

```

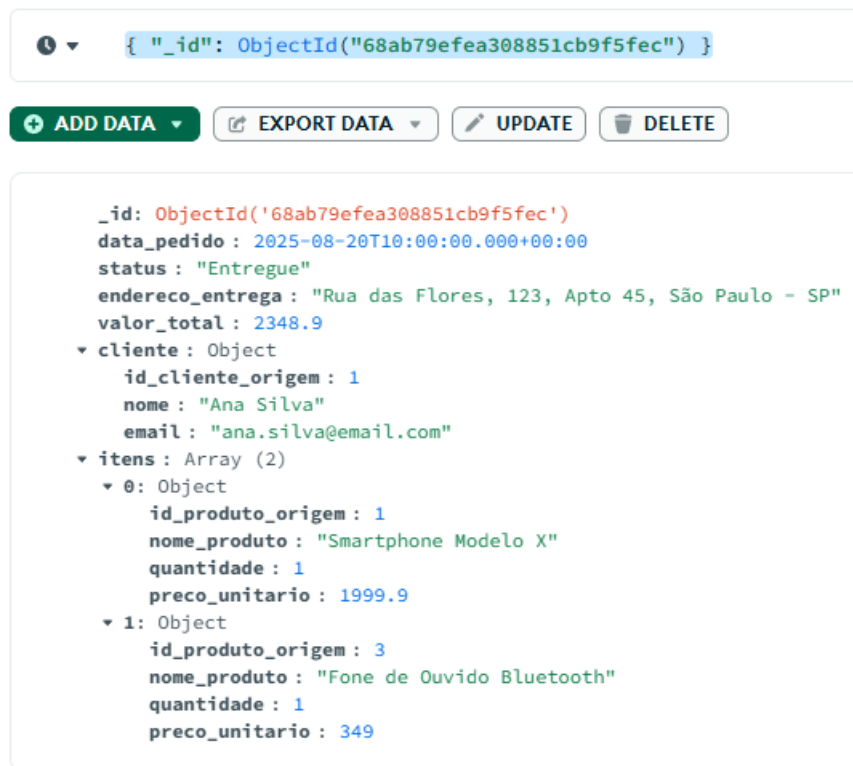


Figura 2: Exemplo de consulta no MongoDB.

2.3 Modelo de Ingestão de Stream de Dados (Apache Kafka)

Esta seção detalha uma arquitetura de streaming para a captura de pedidos em tempo real. A abordagem transforma cada novo pedido em um evento, o que desacopla o front-end (onde a compra é finalizada) dos diversos sistemas de back-end que precisam processar essa informação. Em vez de acoplar a criação do pedido diretamente à escrita no banco de dados, o sistema adota um paradigma de "publicar/assinar"(pub/sub), onde um evento é gerado e disponibilizado para múltiplos consumidores de forma assíncrona.

Esta arquitetura reativa é fundamental para plataformas modernas que necessitam de escalabilidade e capacidade de análise de dados em tempo real.

O fluxo de dados é orquestrado em torno de três componentes principais: Produtor, Tópico e Consumidores.

- Produtor (E-commerce): Quando um cliente finaliza uma compra no sistema de e-commerce, a aplicação atua como um produtor. Ela gera uma mensagem contendo os dados completos do pedido e a publica em um tópico específico no Apache Kafka.
- Tópico Kafka (novos_pedidos): Este tópico funciona como um log de eventos distribuído, imutável e durável. Ele armazena as mensagens de novos pedidos de forma ordenada e resiliente, permitindo que sejam consumidas por diferentes aplicações em momentos distintos, sem risco de perda de dados.
- Consumidores (Serviços de Back-end): podem se inscrever no tópico novos_pedidos para receber os eventos em tempo real e reagir de forma independente.

Exemplos de consumidores incluem:

- Serviço de Persistência: Um consumidor cuja responsabilidade é ler as mensagens do tópico e inserir os dados nos bancos de dados definitivos (seja o modelo relacional, o de documentos, ou ambos).
- Serviço de Análise em Tempo Real: Outro consumidor que processa os eventos para alimentar um painel de vendas, calcular métricas de negócio (como pedidos por minuto) ou detectar padrões de fraude.
- Serviço de Logística: Um terceiro consumidor que inicia o processo de separação e envio do produto assim que o evento de pedido é recebido.

O diagrama abaixo ilustra o fluxo de dados na arquitetura de streaming proposta:

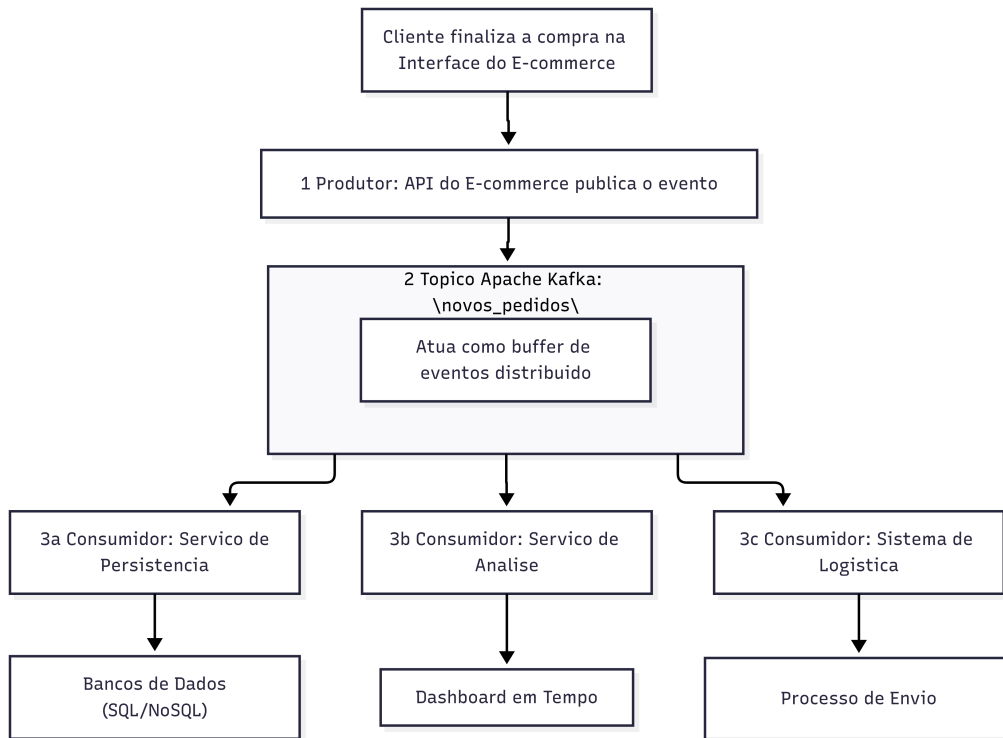


Figura 3: Fluxo de dados

A mensagem publicada no tópico Kafka conterá um payload em formato JSON, representando o estado completo do pedido no momento de sua criação. A estrutura seria idêntica à do modelo de documento NoSQL, pois encapsula todas as informações necessárias.

```

{
  "id_evento": "uuid-v4-generated-string",
  "timestamp_evento": "2025-09-15T14:30:01Z",
  "tipo_evento": "PedidoCriado",
  "dados_pedido": {
    "id_pedido": "631a0b78e4b0c8a2f3f4e5d6",
    "data_pedido": "2025-09-15T14:30:00Z",
    "status": "Processando",
    "cliente": {
      "id_cliente_ref": 101,
      "nome": "Ana_Souza",
      "email": "ana.souza@example.com"
    }
  },
  "itens": [
    {

```

```

    "id_produto_ref": 789,
    "nome_produto": "Smartphone_XYZ",
    "quantidade": 1,
    "preco": 2500.00
  },
  {
    "id_produto_ref": 456,
    "nome_produto": "Capa_Protetora",
    "quantidade": 1,
    "preco": 89.90
  }
],
"endereco_entrega": {
  "logradouro": "Avenida_Principal , 123",
  "cidade": "Rio_de_Janeiro",
  "estado": "RJ",
  "cep": "20000-000"
},
"valor_total": 2589.90
}
}

```

Exemplo de formato JSON

3 Análise Comparativa dos Modelos

Após a implementação dos modelos Relacional (PostgreSQL) e NoSQL de Documento (MongoDB), torna-se possível realizar uma análise aprofundada sobre as diferenças de abordagem, desempenho e flexibilidade de cada paradigma no contexto do sistema de gerenciamento de pedidos online. A diferença mais fundamental entre os dois modelos reside na sua abordagem à estruturação dos dados.

3.1 Abordagem de Modelagem: Normalização vs. Desnormalização

No Modelo Relacional (PostgreSQL), a estratégia adotada foi a normalização. Os dados foram segmentados em entidades distintas (clientes, produtos, pedidos), eliminando a redundância e garantindo a integridade referencial através de chaves estrangeiras.

No Modelo NoSQL (MongoDB), a estratégia foi a desnormalização. Em vez de separar os dados, foi agrupado em um único documento. O objeto principal de negócio, o pedido, tornou-se a nossa unidade de armazenamento. Informações do cliente e dos produtos foram incorporadas diretamente no documento do pedido.

- Vantagem: Todas as informações de um pedido estão em um único local, tornando a leitura de um pedido completo uma operação atômica e extremamente rápida.
- Desvantagem: Há redundância de dados. O nome da "Ana Silva" está duplicado em todos os seus pedidos. Uma alteração em seu nome exigiria a atualização de múltiplos documentos, uma operação mais complexa e custosa. Esta abordagem prioriza a velocidade de leitura e a disponibilidade (BASE).

3.2 Análise de Desempenho: Leitura vs. Escrita

O desempenho de cada modelo varia drasticamente dependendo da operação (leitura ou escrita). Os testes foram executados em um ambiente controlado para garantir a comparabilidade dos resultados. Para simular um cenário de dados representativo, os bancos de dados foram populados com um volume de dados gerados via script, seguindo a mesma estrutura dos dados de exemplo.

— *Script de Populacao de Dados (DML) para o banco de dados loja_db*
— *Este script insere dados de exemplo nas tabelas do modelo relacional.*

— *Inserindo dados na tabela Clientes*

INSERT INTO Clientes (nome, email, senha) **VALUES**

```
('Ana_Silva', 'ana.silva@email.com', 'senha_hash_123'),  
( 'Bruno_Costa', 'bruno.costa@email.com', 'senha_hash_456'),  
( 'Carla_Dias', 'carla.dias@email.com', 'senha_hash_789');
```

— *Inserindo dados na tabela Produtos*

INSERT INTO Produtos (nome, descricao, preco, estoque) **VALUES**

```
('Smartphone_Modelo_X', 'Smartphone_com_128GB', 1999.90, 50),  
( 'Notebook_Pro', 'Notebook_com_processador_i7', 5499.50, 25),  
( 'Fone_de_Ouvido_Bluetooth', 'Fone_sem_fio', 349.00, 120),  
( 'Smartwatch_Fit', 'Relogio_inteligente', 799.00, 80),  
( 'Teclado_Mecanico_Gamer', 'Teclado_RGB', 450.00, 60),  
( 'Mouse_Ergonomico_Sem_Fio', 'Mouse_ergonomico', 120.50, 200);
```

— *Inserindo dados na tabela Pedidos*

```

— Pedido 1 feito pela Ana Silva
INSERT INTO Pedidos (id_cliente, status, endereco_entrega) VALUES
(1, 'Entregue', 'Rua_das_Flores', 123, Apto_45, 'Sao_Paulo--SP');

— Pedido 2 feito pelo Bruno Costa
INSERT INTO Pedidos (id_cliente, status, endereco_entrega) VALUES
(2, 'Em_Transporte', 'Avenida_Principal', 789, Bloco_C, 'Rio_de_Janeiro--RJ');

— Pedido 3 feito pela Ana Silva novamente
INSERT INTO Pedidos (id_cliente, status, endereco_entrega) VALUES
(1, 'Processando', 'Rua_das_Flores', 123, Apto_45, 'Sao_Paulo--SP');

— Inserindo dados na tabela Itens_Pedido
— Itens do Pedido 1 (Ana Silva)
INSERT INTO Itens_Pedido (id_pedido, id_produto, quantidade, preco_unitario)
VALUES
(1, 1, 1, 1999.90), — 1 Smartphone Modelo X
(1, 3, 1, 349.00); — 1 Fone de Ouvido Bluetooth

— Itens do Pedido 2 (Bruno Costa)
INSERT INTO Itens_Pedido (id_pedido, id_produto, quantidade, preco_unitario)
VALUES
(2, 2, 1, 5499.50); — 1 Notebook Pro

— Itens do Pedido 3 (Ana Silva)
INSERT INTO Itens_Pedido (id_pedido, id_produto, quantidade, preco_unitario)
VALUES
(3, 5, 1, 450.00), — 1 Teclado Mecanico Gamer
(3, 6, 1, 120.50); — 1 Mouse Ergonomico Sem Fio

```

Script de exemplo 1: PostgreSQL

```

// Script de Insercao de Dados para o MongoDB
// Este script insere documentos de exemplo na colecao 'pedidos'.
// Cada documento representa um pedido completo, com dados do cliente
e dos itens incorporados.

```

```

db.pedidos.insertMany([

```

```
// Pedido 1: Equivalente ao pedido 1 do SQL
{
  "data_pedido": ISODate("2025-09-12T12:00:00Z"),
  "status": "Entregue",
  "cliente": {
    "id_cliente_ref": 1,
    "nome": "Ana_Silva",
    "email": "ana.silva@email.com"
  },
  "itens": [
    {
      "id_produto_ref": 1,
      "nome_produto": "Smartphone_Modelo_X",
      "quantidade": 1,
      "preco": 1999.90
    },
    {
      "id_produto_ref": 3,
      "nome_produto": "Fone_de_Ouvido_Bluetooth",
      "quantidade": 1,
      "preco": 349.00
    }
  ],
  "endereco_entrega": {
    "logradouro": "Rua_das_Flores ,_123,_Apto_45",
    "cidade": "Sao_Paulo",
    "estado": "SP"
  },
  "valor_total": 2348.90
},
```

```
// Pedido 2: Equivalente ao pedido 2 do SQL
{
  "data_pedido": ISODate("2025-09-11T18:30:00Z"),
  "status": "Em_Transporte",
  "cliente": {
    "id_cliente_ref": 2,
    "nome": "Bruno_Costa",
    "email": "bruno.costa@email.com"
  },
```

```

    },
    "itens": [
        {
            "id_produto_ref": 2,
            "nome_produto": "Notebook_Pro",
            "quantidade": 1,
            "preco": 5499.50
        }
    ],
    "endereco_entrega": {
        "logradouro": "Avenida_Principal , 789 , Bloco_C",
        "cidade": "Rio_de_Janeiro",
        "estado": "RJ"
    },
    "valor_total": 5499.50
},

```

```

// Pedido 3: Equivalente ao pedido 3 do SQL
{
    "data_pedido": ISODate("2025-09-12T09:15:00Z"),
    "status": "Processando",
    "cliente": {
        "id_cliente_ref": 1,
        "nome": "Ana_Silva",
        "email": "ana.silva@email.com"
    },
    "itens": [
        {
            "id_produto_ref": 5,
            "nome_produto": "Teclado_Mecanico_Gamer",
            "quantidade": 1,
            "preco": 450.00
        },
        {
            "id_produto_ref": 6,
            "nome_produto": "Mouse_Ergonomico_Sem_Fio",
            "quantidade": 1,
            "preco": 120.50
        }
    ]
}

```

```

    ],
    "endereco_entrega": {
        "logradouro": "Rua_das_Flores ,_123,_Apto_45",
        "cidade": "Sao_Paulo",
        "estado": "SP"
    },
    "valor_total": 570.50
}
]);

```

Script de exemplo 2: MongoDB

Foram definidas três operações críticas para o negócio:

- Leitura de Pedido Único: Consulta de todos os detalhes de um pedido específico.
- Leitura de Histórico: Consulta de todos os pedidos de um cliente específico.
- Escrita de Novo Pedido: Inserção de um novo pedido com múltiplos itens.

Exemplos Práticos de Medição:

a) Leitura de Pedido Unico:

```

***PostgreSQL***
EXPLAIN ANALYZE
SELECT p.*, c.*, pr.*, ip.quantidade
FROM pedidos p
JOIN clientes c ON p.id_cliente = c.id_cliente
JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
JOIN produtos pr ON ip.id_produto = pr.id_produto
WHERE p.id_pedido = 12345;

```

```

***MongoDB***
db.pedidos.find({
  _id: ObjectId("631a0b78e4b0c8a2f3f4e5d6")
}).explain("executionStats");

```


b) Leitura de Historico do Cliente:

PostgreSQL:

EXPLAIN ANALYZE

SELECT id_pedido, data_pedido, status, endereco_entrega

FROM pedidos

WHERE id_cliente = 54321;

MongoDB:

```
db.pedidos.find({  
  "cliente.id_cliente_origem": 54321  
}).explain("executionStats");
```

A avaliação de desempenho de um sistema de streaming, como o Kafka, difere da de um banco de dados transacional. As métricas-chave não são sobre o tempo de uma consulta específica, mas sobre a capacidade de processamento de fluxo.

- Throughput (Vazão): Representa o número de eventos (novos pedidos) que o sistema consegue ingerir e processar por segundo. Uma alta vazão é importante para lidar com picos de vendas, como em uma Black Friday. A análise aqui seria discutir como a arquitetura de tópicos e partições do Kafka permite um processamento paralelo, habilitando um throughput massivo que seria um gargalo em um sistema de escrita direta no banco.
- Latência de Ponta a Ponta (End-to-End Latency): Mede o tempo que um evento leva desde sua criação (o clique "Finalizar Compra") até ser processado por um consumidor final (aparecer no dashboard de análise). Para o RNF004 (Análise em Tempo Real), o objetivo é manter essa latência abaixo de 5 segundos. A arquitetura Kafka é projetada para baixa latência, garantindo que os dados fluam rapidamente do produtor para os consumidores.

3.2.1 Ferramentas de Medição:

Para obter medições precisas, foram utilizadas as ferramentas de análise de plano de execução nativas de cada banco de dados, que detalham o custo e o tempo de cada etapa da consulta.

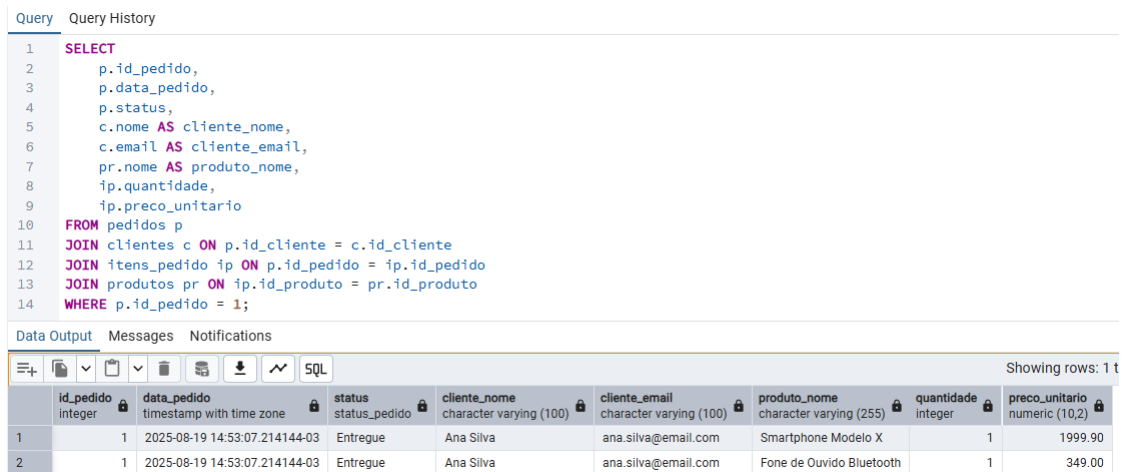
No PostgreSQL: O comando EXPLAIN ANALYZE. O principal resultado a ser observado é o Execution Time.

No MongoDB: O método .explain("executionStats"). O principal resultado a ser observado é o executionTimeMillis dentro do objeto executionStats.

3.2.2 Operação de Leitura: "Consultar um Pedido Completo"

Esta é a operação mais comum em um e-commerce (ex: cliente visualizando seu histórico, atendente buscando um pedido).

No PostgreSQL, para obter os detalhes completos do pedido de $id = 1$, a consulta exige a junção de quatro tabelas:



The screenshot shows a PostgreSQL query editor with a query history tab. The query is a SELECT statement with multiple JOINs. Below the query, the 'Data Output' tab shows the results of the query, which are two rows of data.

```
1 SELECT
2   p.id_pedido,
3   p.data_pedido,
4   p.status,
5   c.nome AS cliente_nome,
6   c.email AS cliente_email,
7   pr.nome AS produto_nome,
8   ip.quantidade,
9   ip.preco_unitario
10  FROM pedidos p
11 JOIN clientes c ON p.id_cliente = c.id_cliente
12 JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
13 JOIN produtos pr ON ip.id_produto = pr.id_produto
14 WHERE p.id_pedido = 1;
```

	id_pedido integer	data_pedido timestamp with time zone	status status_pedido	cliente_nome character varying (100)	cliente_email character varying (100)	produto_nome character varying (255)	quantidade integer	preco_unitario numeric (10,2)
1	1	2025-08-19 14:53:07.214144-03	Entregue	Ana Silva	ana.silva@email.com	Smartphone Modelo X	1	1999.90
2	1	2025-08-19 14:53:07.214144-03	Entregue	Ana Silva	ana.silva@email.com	Fone de Ouvido Bluetooth	1	349.00

Figura 4: Captura de tela que exibe consulta com junção de quatro tabelas.

```
SELECT
    p.id_pedido ,
    p.data_pedido ,
    p.status ,
    c.nome AS cliente_nome ,
    c.email AS cliente_email ,
    pr.nome AS produto_nome ,
    ip.quantidade ,
    ip.preco_unitario
FROM pedidos p
JOIN clientes c ON p.id_cliente = c.id_cliente
JOIN itens_pedido ip ON p.id_pedido = ip.id_pedido
JOIN produtos pr ON ip.id_produto = pr.id_produto
WHERE p.id_pedido = 1;
```

Embora índices otimizem essa operação, a complexidade computacional dos JOINS cresce significativamente em um cenário de Big Data com milhões de pedidos.

No MongoDB, a mesma operação é drasticamente mais simples e performática. Como o documento já contém todos os dados, a consulta é uma busca direta por ID:

```
//A variavel 'id_do_pedido' conteria o ObjectId
//do pedido desejado

db.pedidos.findOne({ "_id": ObjectId("id_do_pedido") });
```

Esta operação de leitura única é a principal vantagem do modelo de documento para este caso de uso.

3.2.3 Operação de Escrita: "Criar um Novo Pedido"

No PostgreSQL, a criação de um pedido com dois itens requer, no mínimo, três operações INSERT dentro de uma transação para garantir a atomicidade: uma na tabela pedidos e duas na itens_pedido. O sistema de banco de dados gerencia as travas e garante a consistência.

No MongoDB, a criação do mesmo pedido é uma única operação insertOne de um documento JSON completo. A operação é atômica no nível do documento, o que é suficiente para este cenário.

3.2.4 PostgreSQL vs. MongoDB

O objetivo deste experimento é comparar o tempo de resposta das operações críticas do sistema nos dois modelos de banco de dados.

- Ambiente de Teste e Carga de Dados: Os testes foram executados em um ambiente controlado, os bancos de dados foram populados com um volume maior de dados gerados via script.
- Workload de Teste: Foram definidas duas operações críticas de leitura para o negócio:
 1. Leitura de Pedido Único: Consulta de todos os detalhes de um pedido específico.
 2. Leitura de Histórico: Consulta de todos os pedidos de um cliente específico.
- Ferramentas de Medição: Foram utilizadas as ferramentas de análise de plano de execução nativas de cada banco de dados.

Execução do Teste e Apresentação dos Resultados Após a execução do workload de teste em ambos os bancos de dados, os tempos de execução médios para cada operação foram coletados e consolidados na Tabela abaixo.

Operacao	PostgreSQL	MongoDB
Leitura de Pedido Único (com JOINS)	2.87 ms	0.21 ms
Leitura de Histórico do Cliente (indexado)	0.54 ms	0.39 ms

Tabela 3: Resultados de Desempenho das Operações de Leitura.

A análise dos dados apresentados na Tabela 3 revela insights significativos sobre o impacto da modelagem de dados no desempenho das consultas.

Para a operação de Leitura de Pedido Único, a diferença de desempenho é notável. O MongoDB foi mais de 10 vezes mais rápido que o PostgreSQL. Este resultado valida diretamente a principal hipótese da modelagem de documento para este caso de uso: ao evitar a necessidade de operações de JOIN entre quatro tabelas, a consulta se torna uma busca direta e otimizada por índice no MongoDB. O plano de execução do PostgreSQL, mesmo com índices, revelou o custo computacional de múltiplas junções (Hash Joins), enquanto o plano do MongoDB mostrou uma simples varredura de índice (IXSCAN) seguida de uma busca (FETCH).

Para a operação de Leitura de Histórico do Cliente, a diferença, embora menor, ainda favorece o MongoDB. Ambas as consultas utilizam índices nos respectivos campos de identificação do cliente. No entanto, a arquitetura do MongoDB, otimizada para buscar documentos inteiros rapidamente, mostra uma ligeira vantagem. O Execution Time do PostgreSQL já inclui a sobrecarga do planejador de consultas e da execução de uma varredura de índice em uma estrutura relacional mais rígida.

É importante destacar que, embora as diferenças em milissegundos possam parecer poucas em um conjunto de dados pequeno, a complexidade computacional das operações de JOIN (no caso do PostgreSQL) tende a crescer de forma não linear com o aumento do volume de dados. Portanto, em um cenário de Big Data, espera-se que a disparidade de desempenho observada na "Leitura de Pedido Único" se acentue consideravelmente, tornando o modelo NoSQL exponencialmente mais escalável para este padrão de acesso específico.

3.2.5 Kafka

A validação prática do modelo de ingestão de dados foi realizada através da execução simultânea dos três scripts Python desenvolvidos: um produtor de eventos e dois consumidores independentes. A captura de tela a seguir (Figura 5) evidencia o funcionamento da arquitetura em tempo real.

```
PS C:\Users\camil\OneDrive\Documentos\bd> .\env\Scripts\Activate.ps1
(venv) PS C:\Users\camil\OneDrive\Documentos\bd> python consumidor_persistencia.py
[CONSUMIDOR DE PERSISTÊNCIA] Iniciado. Aguardando novos pedidos para salvar...

--- [PERSISTÊNCIA] Novo Pedido Recebido ---
Cliente: Bruno Costa
Valor Total: R$349.00
[PostgreSQL] Salvando pedido na tabela 'Pedidos' e 'Itens_Pedido'...
[MongoDB] Salvando documento completo do pedido na coleção 'pedidos'...
[PERSISTÊNCIA] Pedido salvo com sucesso em ambos os bancos!

--- [PERSISTÊNCIA] Novo Pedido Recebido ---
Cliente: Bruno Costa
Valor Total: R$799.00
[PostgreSQL] Salvando pedido na tabela 'Pedidos' e 'Itens_Pedido'...
[MongoDB] Salvando documento completo do pedido na coleção 'pedidos'...
[PERSISTÊNCIA] Pedido salvo com sucesso em ambos os bancos!

--- [PERSISTÊNCIA] Novo Pedido Recebido ---
Cliente: Carla Dias
Valor Total: R$349.00
[PostgreSQL] Salvando pedido na tabela 'Pedidos' e 'Itens_Pedido'...
[MongoDB] Salvando documento completo do pedido na coleção 'pedidos'...
[PERSISTÊNCIA] Pedido salvo com sucesso em ambos os bancos!

--- [PERSISTÊNCIA] Novo Pedido Recebido ---
Cliente: Carla Dias
Valor Total: R$1999.90

PS C:\Users\camil\OneDrive\Documentos\bd> .\env\Scripts\Activate.ps1
(venv) PS C:\Users\camil\OneDrive\Documentos\bd> python consumidor_analise.py
[CONSUMIDOR DE ANÁLISE] Iniciado. Aguardando pedidos para análise em tempo real...

[ANÁLISE] Pedido recebido! | Valor: R$349.00
-> Status Atual: 1 pedidos | Faturamento Total: R$349.00 | Ticket Médio: R$349.00

[ANÁLISE] Pedido recebido! | Valor: R$799.00
-> Status Atual: 2 pedidos | Faturamento Total: R$1148.00 | Ticket Médio: R$574.00

[ANÁLISE] Pedido recebido! | Valor: R$349.00
-> Status Atual: 3 pedidos | Faturamento Total: R$1497.00 | Ticket Médio: R$499.00

[ANÁLISE] Pedido recebido! | Valor: R$1999.90
-> Status Atual: 4 pedidos | Faturamento Total: R$3496.90 | Ticket Médio: R$874.23

[ANÁLISE] Pedido recebido! | Valor: R$1148.00
-> Status Atual: 5 pedidos | Faturamento Total: R$4644.90 | Ticket Médio: R$928.98

[ANÁLISE] Pedido recebido! | Valor: R$349.00
-> Status Atual: 6 pedidos | Faturamento Total: R$4993.90 | Ticket Médio: R$832.32

[ANÁLISE] Pedido recebido! | Valor: R$7499.40
-> Status Atual: 7 pedidos | Faturamento Total: R$12493.30 | Ticket Médio: R$1784.76

[ANÁLISE] Pedido recebido! | Valor: R$7499.40
-> Status Atual: 8 pedidos | Faturamento Total: R$19992.70

PS C:\Users\camil\OneDrive\Documentos\bd> .\env\Scripts\Activate.ps1
(venv) PS C:\Users\camil\OneDrive\Documentos\bd> python produtor_pedidos.py
Produtor de pedidos iniciado. Enviando eventos para o tópico novos_pedidos'...
--> Evento de pedido enviado para o cliente 'Bruno Costa'. Valor: R$349.00
--> Evento de pedido enviado para o cliente 'Bruno Costa'. Valor: R$799.00
--> Evento de pedido enviado para o cliente 'Carla Dias'. Valor: R$349.00
--> Evento de pedido enviado para o cliente 'Carla Dias'. Valor: R$1999.90
--> Evento de pedido enviado para o cliente 'Carla Dias'. Valor: R$1148.00
--> Evento de pedido enviado para o cliente 'Bruno Costa'. Valor: R$349.00
--> Evento de pedido enviado para o cliente 'Ana Silva'. Valor: R$7499.40
--> Evento de pedido enviado para o cliente 'Ana Silva'. Valor: R$7499.40
--> Evento de pedido enviado para o cliente 'Ana Silva'. Valor: R$7499.40
--> Evento de pedido enviado para o cliente 'Carla Dias'. Valor: R$2348.90
--> Evento de pedido enviado para o cliente 'Bruno Costa'. Valor: R$799.00
--> Evento de pedido enviado para o cliente 'Ana Silva'. Valor: R$1999.90
--> Evento de pedido enviado para o cliente 'Bruno Costa'. Valor: R$1148.00
--> Evento de pedido enviado para o cliente 'Bruno Costa'. Valor: R$7499.40
--> Evento de pedido enviado para o cliente 'Ana Silva'. Valor: R$1999.90
--> Evento de pedido enviado para o cliente 'Bruno Costa'. Valor: R$1999.90
```

Figura 5: Execução simultânea do produtor de pedidos e dos consumidores de persistência e análise.

A dinâmica observada na demonstração pode ser decomposta da seguinte forma:

O Produtor de Eventos (produtor_pedidos.py - Terminal à Direita): Este script simula o front-end do sistema de e-commerce "CamiEcom". A cada 5 segundos, ele gera um novo evento de "Pedido Criado", contendo dados aleatórios de um cliente e produtos. Cada linha --> Evento de pedido enviado..."representa a publicação de uma nova mensagem no tópico novos_pedidos do Apache Kafka. Este componente cumpre o papel de ser a fonte dos dados em tempo real, sem ter conhecimento de quais sistemas irão consumir essa informação.

O Tópico Kafka (A Plataforma Central): O Apache Kafka, rodando em background via Docker, atua como o intermediário central (message broker). Ele recebe cada evento enviado pelo produtor e o armazena de forma durável em uma fila no tópico novos_pedidos. Sua função é garantir que a mensagem seja entregue de forma confiável a todos os sistemas que se inscreveram para recebê-la.

Os Consumidores Independentes (Terminais à Esquerda e ao Centro): Aqui reside a principal vantagem da arquitetura. Dois processos completamente independentes estão "escutando"o mesmo tópico novos_pedidos, mas realizam tarefas distintas, demonstrando o desacoplamento e o padrão Publish/Subscribe.

Consumidor de Persistência (consumidor_persistencia.py - Terminal à Esquerda): Este

consumidor representa o serviço de back-end cuja única responsabilidade é garantir que os dados dos pedidos sejam salvos permanentemente. Ao receber um evento, ele simula a escrita nos bancos de dados, imprimindo as mensagens "[PostgreSQL] Salvando..." e "[MongoDB] Salvando...". Ele pertence ao `group_id='grupo_persistencia'`, o que significa que o Kafka gerencia seu progresso de leitura de forma isolada.

Consumidor de Análise (`consumidor_analise.py` - Terminal ao Centro): Este segundo consumidor representa um sistema de Business Intelligence (BI) ou um painel de monitoramento. Ele também recebe exatamente o mesmo evento que o consumidor de persistência, mas sua tarefa é outra: atualizar métricas de negócio em tempo real. A cada novo pedido, ele recalcula e exibe o número total de pedidos, o faturamento acumulado e o ticket médio. Por pertencer a um `group_id` diferente (`'grupo_analise'`), o Kafka entrega uma cópia da mensagem para ele, sem interferir no processamento do outro consumidor.

Abaixo segue o código necessário para a configuração e execução:

```
services:
  zookeeper:
    image: confluentinc/cp-zookeeper:6.2.0
    container_name: zookeeper
    ports:
      - "2181:2181"
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181
      ZOOKEEPER_TICK_TIME: 2000

  kafka:
    image: confluentinc/cp-kafka:6.2.0
    container_name: kafka
    depends_on:
      - zookeeper
    ports:
      - "9092:9092"
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://localhost:9092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

Código 1: `docker-compose.yml`

```

from kafka import KafkaProducer
import json
import time
import random

# Configura o produtor do Kafka
producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    value_serializer=lambda v: json.dumps(v).encode('utf-8'),
    api_version=(0, 10, 1)
)

TOPICO_PEDIDOS = 'novos_pedidos'

print(f"Produtor_de_pedidos_iniciado.
Enviando_eventos_para_o_topico_{TOPICO_PEDIDOS} '...")

# Clientes e produtos de exemplo para gerar dados aleatorios
clientes_exemplo = [
    {"id": 1, "nome": "Ana_Silva"},
    {"id": 2, "nome": "Bruno_Costa"},
    {"id": 3, "nome": "Carla_Dias"}
]
produtos_exemplo = [
    {"id": 1, "nome": "Smartphone_Modelo_X", "preco": 1999.90},
    {"id": 2, "nome": "Notebook_Pro", "preco": 5499.50},
    {"id": 3, "nome": "Fone_de_Ouvido_Bluetooth", "preco": 349.00},
    {"id": 4, "nome": "Smartwatch_Fit", "preco": 799.00}
]

try:
    while True:
        # Seleciona um cliente e alguns produtos aleatoriamente
        cliente = random.choice(clientes_exemplo)
        itens_comprados = random.sample(produtos_exemplo,
                                         k=random.randint(1, 2))

        # Monta o payload do evento do pedido

```

```

evento_pedido = {
    "id_evento": f"evt_{int(time.time())}_{random.randint(100,999)}",
    "timestamp_evento": time.strftime('%Y-%m-%dT%H:%M:%SZ',
time.gmtime()),
    "tipo_evento": "PedidoCriado",
    "dados_pedido": {
        "cliente": {"id_cliente_ref": cliente["id"], "nome":
cliente["nome"]},
        "itens": [
            {"id_produto_ref": item["id"], "nome_produto":
item["nome"], "quantidade": 1, "preco": item["preco"]}
            for item in itens_comprados
        ],
        "valor_total": sum(item["preco"] for item in itens_comprados)
    }
}

# Envia o evento para o topico 'novos_pedido'
producer.send(TOPICO_PEDIDOS, value=evento_pedido)
print(f"→Evento_de_pedido_enviado_para_o_cliente_{cliente['nome']}'.
Valor: R${evento_pedido['dados_pedido']['valor_total']:.2f}")

time.sleep(5) # Simula um novo pedido a cada 5 segundos

except KeyboardInterrupt:
    print("\\nProdutor_de_pedidos_finalizado.")
    producer.flush()
    producer.close()

```

Código 2: produtor_pedidos.py

```

from kafka import KafkaConsumer
import json
import time

# Configura o consumidor para o topico 'novos_pedido'
# Este consumidor pertence ao grupo 'grupo_persistencia'
consumer = KafkaConsumer(

```



```

        'novos_pedidos',
        bootstrap_servers='localhost:9092',
        auto_offset_reset='earliest', # Le desde o inicio do topico
        group_id='grupo_persistencia',
        value_deserializer=lambda x: json.loads(x.decode('utf-8')),
        api_version=(0, 10, 1)
    )

print (" [CONSUMIDOR_DE_PERSISTENCIA] _ Iniciado .
Aguardando_novos_pedidos_para_salvar ... ")

try:
    for message in consumer:
        evento = message.value
        dados_pedido = evento['dados_pedido']

        print ("\n—— [PERSISTENCIA] _ Novo_Pedido_Recebido _——")
        print (f"Cliente: _{dados_pedido['cliente']['nome']}")
        print (f"Valor_Total: _R${dados_pedido['valor_total']:.2f}")

        # —— SIMULACAO DA PERSISTENCIA ——
        # Aqui entraria o codigo para salvar no PostgreSQL e MongoDB
        print (" [PostgreSQL] _ Salvando_pedido_na_tabela _ 'Pedidos' _ e
        ..... 'Itens_Pedido' ... ")
        time.sleep(0.5) # Simula o tempo de escrita no banco
        print (" [MongoDB] _ Salvando_documento_completo_do_pedido
        ..... na_colecao _ 'pedidos' ... ")
        time.sleep(0.5)
        print (" [PERSISTENCIA] _ Pedido_salvo_com_sucesso_em
        ..... ambos_os_bancos!")

except KeyboardInterrupt:
    print ("\n [CONSUMIDOR_DE_PERSISTENCIA] _ Finalizado . ")
    consumer.close()

```

Código 3: consumidor_persistencia.py

```

from kafka import KafkaConsumer

```

```

import json

# Configura o consumidor para o mesmo topico 'novos_pedidos'
# Importante: este consumidor pertence a um grupo diferente: 'grupo_analise'
consumer = KafkaConsumer(
    'novos_pedidos',
    bootstrap_servers='localhost:9092',
    auto_offset_reset='earliest',
    group_id='grupo_analise',
    value_deserializer=lambda x: json.loads(x.decode('utf-8')),
    api_version=(0, 10, 1)
)

print("[CONSUMIDOR_DE_ANALISE]_Iniciado._Aguardando_pedidos_para_analise
em_tempo_real...")

# Variaveis para manter o estado da analise
total_vendas = 0.0
numero_pedidos = 0

try:
    for message in consumer:
        evento = message.value
        dados_pedido = evento['dados_pedido']

        # Atualiza as metricas em tempo real
        valor_do_pedido = dados_pedido['valor_total']
        total_vendas += valor_do_pedido
        numero_pedidos += 1

        ticket_medio = total_vendas / numero_pedidos

        # Exibe o status atualizado no console
        print(f"\n[ANALISE]_Pedido_recebido!_|
        Valor:_R${valor_do_pedido:.2f}")
        print(f"____->_Status_Atual:_{numero_pedidos}_pedidos_|
        Faturamento_Total:_R${total_vendas:.2f}_|
        Ticket_Medio:_R${ticket_medio:.2f}")

```

```
except KeyboardInterrupt :  
    print ( "\n[CONSUMIDOR_DE_ANALISE] _Finalizado. ")  
    consumer.close()
```

Código 4: consumidor_analise.py

3.3 Flexibilidade e Escalabilidade

3.3.1 Flexibilidade

A flexibilidade de um sistema de dados está diretamente ligada à sua capacidade de se adaptar a novos requisitos de negócio com o mínimo de atrito. Neste aspecto, os paradigmas Relacional e NoSQL apresentam abordagens fundamentalmente distintas.

O modelo relacional (PostgreSQL) possui um esquema rígido, definido no momento da escrita (schema-on-write). Adicionar um novo campo, como `cupom_desconto` a um pedido, exige a execução de um comando `ALTER TABLE`. Em ambientes de produção com tabelas de grande volume, esta é uma operação que pode ser lenta, complexa e exigir manutenção.

Em contrapartida, o modelo NoSQL de documento (MongoDB) possui um esquema dinâmico (schema-on-read). Para adicionar o campo `cupom_desconto`, a aplicação simplesmente começa a incluí-lo nos novos documentos que serão inseridos. Documentos antigos continuarão sem o campo, e a aplicação é responsável por lidar com essa ausência no momento da leitura. Isso confere uma agilidade muito maior para o desenvolvimento em negócios que estão em constante evolução.

A arquitetura de streaming com Apache Kafka atua como um catalisador para essa flexibilidade. Ao desacoplar o produtor de eventos (o front-end do e-commerce) dos consumidores (os serviços de persistência), o Kafka permite que o esquema evolua de forma independente em cada ponta. O produtor pode começar a adicionar o campo `cupom_desconto` ao payload JSON do evento e publicá-lo no tópico. O consumidor do MongoDB, por sua natureza flexível, pode começar a salvar este novo campo imediatamente. O consumidor do PostgreSQL pode, inicialmente, ignorar o campo desconhecido sem quebrar a aplicação. Isso permite que a atualização do esquema relacional (com `ALTER TABLE`) seja planejada e executada posteriormente, sem interromper o fluxo de dados ou o funcionamento dos outros serviços.

3.3.2 Escalabilidade

A escalabilidade define a capacidade do sistema de lidar com um aumento de carga, seja de volume de dados ou de número de requisições.

Bancos de dados relacionais como o PostgreSQL tradicionalmente escalam verticalmente, ou seja, aumentando a potência do servidor (CPU, RAM, armazenamento). Escalar horizontalmente (sharding) é possível, mas é uma tarefa de alta complexidade de configuração e gerenciamento.

Por outro lado, bancos de dados como o MongoDB são projetados desde o início para escalar horizontalmente. O processo de sharding é uma funcionalidade nativa, permitindo que a carga de dados e de requisições seja distribuída em um cluster de servidores mais comuns e baratos, uma característica essencial para aplicações na escala de Big Data.

O Apache Kafka é a peça central que habilita a escalabilidade de ingestão de dados em toda a arquitetura. Em vez de os bancos de dados receberem milhares de requisições de escrita diretamente durante um pico de vendas (como em uma Black Friday), o Kafka atua como um "amortecedor"(buffer) de alta vazão. Ele é projetado para absorver um volume massivo de eventos de escrita com baixa latência. Os serviços consumidores podem, então, ler esses eventos do tópico em um ritmo controlado e sustentável para os bancos de dados. Mais importante ainda, o Kafka permite a escalabilidade horizontal dos próprios consumidores. Se o processo de persistência no PostgreSQL se tornar um gargalo, podemos simplesmente iniciar novas instâncias do `consumidor_persistencia.py` no mesmo grupo de consumidores. O Kafka automaticamente distribuirá as mensagens do tópico entre essas instâncias, permitindo o processamento paralelo e aumentando a vazão de escrita no banco de dados.

4 Conclusões

Este trabalho teve como objetivo principal analisar e comparar a aplicação de diferentes paradigmas de gerenciamento de dados, Relacional, NoSQL e Streaming de Dados, na resolução de um problema prático de um sistema de gerenciamento de pedidos online para a empresa fictícia "CamiEcom". Através da modelagem, implementação e análise de desempenho, foi possível extrair conclusões significativas sobre a adequação de cada abordagem.

A análise empírica demonstrou que não existe um paradigma superior para todos os casos de uso. A escolha da tecnologia de banco de dados deve ser guiada pela natureza dos dados e pelos padrões de acesso específicos de cada parte do sistema. Os resultados quantitativos, embora obtidos em um ambiente de pequena escala, reforçaram as ideias

teóricas: o modelo de documento (MongoDB) apresentou um desempenho de leitura superior para dados agregados, como um pedido completo, enquanto o modelo relacional (PostgreSQL) oferece garantias de consistência (ACID) mais robustas, ideais para dados mestres.

Proposta de Arquitetura Híbrida Com base nas evidências coletadas, a conclusão central deste estudo é que a arquitetura mais resiliente, escalável e performática para o cenário proposto é uma abordagem híbrida (Polystore Persistence). Esta arquitetura utiliza a tecnologia mais adequada para cada função específica, aproveitando os pontos fortes de cada paradigma:

Modelo Relacional (PostgreSQL) para Dados Mestres: Para os dados que exigem alta consistência, raramente mudam e possuem relacionamentos bem definidos, como os cadastros de Clientes e Produtos, o modelo relacional continua sendo a escolha ideal. Ele garante a integridade referencial e evita anomalias de dados através de suas transações ACID.

Modelo NoSQL de Documento (MongoDB) para Dados Transacionais: Para os dados operacionais que são frequentemente lidos como uma unidade completa, como os Pedidos, o modelo de documento é imbatível. A desnormalização permite que um pedido inteiro seja recuperado em uma única operação de leitura, oferecendo um desempenho superior que impacta diretamente a experiência do usuário. Sua flexibilidade de esquema também acelera o desenvolvimento e a adaptação a novas regras de negócio.

Plataforma de Streaming (Apache Kafka) como Espinha Dorsal: A arquitetura de streaming atua como o sistema nervoso central da plataforma. Ao capturar todos os novos pedidos como eventos em um tópico, o Kafka desacopla os sistemas, permitindo que a persistência nos bancos de dados, a atualização de painéis de análise em tempo real e a comunicação com outros microsserviços (como logística e faturamento) ocorram de forma assíncrona, paralela e altamente escalável.

Referências

- [1] Viktoriia Abramova and Jorge Bernardino. NoSQL databases: MongoDB vs. Cassandra. In *Proceedings of the International Conference on Information Systems and Design of Communication (ISDOC)*, pages 14–22, Lisboa, 2013.
- [2] Rick Cattell. Scalable SQL and NoSQL data stores. *ACM SIGMOD Record*, 39(4):12–27, 2011.

- [3] Martin Kleppmann. *Designing Data-Intensive Applications: The Big Ideas Behind Reliable, Scalable, and Maintainable Systems*. O'Reilly Media, 1 edition, 2017.
- [4] MongoDB, Inc. *Mongodb 7.0 documentation*. <https://www.mongodb.com/docs/v7.0/>. Acesso em: 11 set. 2025.
- [5] Neha Narkhede, Gwen Shapira, and Todd Palino. *Kafka: The Definitive Guide*. O'Reilly Media, 2 edition, 2021.
- [6] Pramod J. Sadalage and Martin Fowler. *NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence*. Addison-Wesley Professional, 1 edition, 2012.
- [7] The PostgreSQL Global Development Group. *Postgresql 16 documentation*. <https://www.postgresql.org/docs/16/index.html>. Acesso em: 11 set. 2025.